

湖北第二师范学院计算机学院

专业实训日报

【2025-2026 第二学期】

课程名称	专业实训 V	班级	计算机科学与技术 XX 班
团队成员	成员A、成员B		
团队组长	成员A	学号	2024XXXXXXXX
项目名称	基于出租车GPS数据的时空特征提取及可视化		
报告日期	2025 年 XX 月 XX 日		

一、团队报告

1、本日进展

本日完成了 OD 表的最终验证与异常订单清洗，并建立了完整的车辆轨迹缓存和统计缓存体系，为后续所有地图页面（OD流向、轨迹回放、交互热力图、路网拥堵）提供了稳定的数据源。实际产出如下：

- OD 表验证与输出：**基于 clean.csv 完成 OD 提取后，对 484,208 个初始 OD 对进行异常过滤——删除距离 ≤ 0 km 的无效订单 1,810 条，最终输出 orders.csv 共 **482,398** 个有效出行对。字段包含车辆 ID、起终点时间/经纬度、OD时长、OD距离等 15 个字段。
- 车辆缓存构建：**采用流式分块（chunksize=500,000）读取 clean.csv，按车辆 ID 分组拼接轨迹，输出 vehicle_data.json（~**2.6 GB**，38,694,096 个轨迹点，11,068 辆车）。同时建立 vehicles.json 偏移索引（O(1) 定位）和 vehicle_list.json 车辆 ID 列表。
- 统计缓存构建：**聚合生成 stats_summary.json，包含 24 小时订单分布（最高峰 22 时 29,714 单）、小时级载客率（峰值 21 时 64.82%）、小时级平均速度（低谷 18 时 27.85 km/h）、Top 20 上下客热点区域（罗湖火车站地区上车最密集，4,038 单）。
- 缓存设计与规范：**所有缓存文件命名、目录结构、字段格式已固定，后续页面（06-10）统一通过 data/cache/ 读取，无需重复扫描 clean.csv。

2、成员分工

成员	主要工作
成员A (组长)	OD 表异常过滤与终版输出；车辆缓存构建（流式分块读取、跨 chunk 车辆拼接、vehicle_data.json 生成、偏移元数据计算）；缓存目录与文件命名规范设计
成员B	统计缓存构建（订单时段分布、载客率、平均速度、上下客热点聚合）；缓存验证（随机车辆轨迹对比、统计缓存与 orders.csv 交叉校验）；stats_summary.json 字段设计

3、问题汇报

问题	状态	说明
OD 距离为 0 的无效订单	已解决	部分配对订单因同一地点立即下车或 GPS 精度问题导致经纬度未变化，OD_Dis_km ≤ 0 共 1,810 条，已过滤
车辆数据跨 chunk 边界	已解	同一辆车的的历史数据可能跨越两个 chunk，采用 buffer_traj 暂存上一 chunk 末尾车辆轨迹，待下一 chunk 继续拼接后再写入

	决	
2.6 GB 缓存无法整体加载	已解决	vehicle_data.json 设计为行流式 JSON 格式，下游脚本通过正则定位车辆键的字节偏移后读取片段，不整体 json.load
元数据偏移精确性	已解决	在写入前记录 point_offset，确保 vehicles.json 中的偏移量与实际写入字节严格一致

二、成员报告

1、成员A（组长）

(1) 本日进展

- 完成 OD 表异常过滤：从 484,208 个初始 OD 对中，过滤 $OD_TIME_s \leq 0$ 的订单 0 条、 $OD_Dis_km \leq 0$ 的订单 1,810 条，终版 orders.csv 含 482,398 个有效出行对。
- 完成 cache/builder.py 中 build_vehicle_cache() 函数的编码与调试，主要设计如下：
 - 流式读取**：使用 `pd.read_csv(chunksize=500_000)` 分块读取 clean.csv，每块按 id 分组，提取 [time, long, lati, status, speed] 五列。
 - 跨 chunk 拼接**：维护 buffer_traj 暂存上一 chunk 末尾未写完的车辆轨迹。当下一 chunk 的首辆车 ID 变化时，先写入上一辆完整轨迹再开始新车辆。
 - 元数据记录**：每写完一辆车即记录 [point_count, offset, 'vehicle_data.json'] 到 vehicle_info 字典，其中 offset 为累积轨迹点数（非字节偏移），供后续 O(1) 定位。
 - 写入格式**：vehicle_data.json 按 {"vid": [[lon, lat, status, speed, hour], ...]} 格式流式写入，手工管理 first_entry 标志控制逗号分隔，避免将 2.6 GB 数据全部加载到内存。
 - 输出文件**：vehicle_data.json (~2.6 GB, 38,694,096 点)、vehicles.json (~515 KB, 11,068 辆车的元数据)、vehicle_list.json (~76 KB, 车辆 ID 有序列表)。
- 所有路径通过 config.py 的 DATA_DIR 常量配置，代码不含硬编码绝对路径。

(2) 问题汇报

- 流式写入 JSON 时需手动管理 first_entry 标志和逗号分隔，初始版本在第二辆车开始缺少前导逗号导致 JSON 格式错误，加 `if not first_entry: f.write(',\n')` 修复。
- 跨 chunk 拼接的关键在于 prev_vid 的比较逻辑：必须在 chunk 内部按 `groupby('id', sort=False)` 迭代完成后，再检查 prev_vid 与下一 chunk 的首 id 是否一致。调试阶段发现按 `sort=True` 分组后顺序被打乱，调整为 `sort=False` 保持原始出现顺序。
- 2.6 GB 文件写完后需验证 JSON 完整性：用 `python -c "import json; json.load(open('vehicle_data.json'))"` 确认文件闭合正确，首次运行因末尾缺少 } 报错，补全 `vd_file.write('\n')` 修复。

2、成员B

(1) 本日进展

- 完成 cache/builder.py 中 build_order_stats() 函数的编码与运行验证，实际输出如下：
 - 订单时段分布**：读取 orders.csv，将开始时间转为 datetime 后按小时分组统计，输出 24 小时订单分布。结果显示：订单量最低为凌晨 5 时 (5,416 单)，最高为 22 时 (29,714 单)，早晚高峰分别出现在 8 时 (23,954 单) 和 20-22 时 (28,174-29,714 单)。
 - 载客率统计**：在 build_vehicle_cache() 中同步维护 hourly_occupied 和 hourly_total 计数器，计算各小时载客率。低谷在凌晨 4 时 (16.77%)，峰值在 21 时 (64.82%)，与订单量分布高度一致。

- **速度统计**：按小时统计 speed > 0 的有效车速记录，计算平均速度。夜间凌晨 5 时速度最高（40.94 km/h），晚高峰 18 时速度最低（27.85 km/h），反映了城市道路的实际拥堵规律。
- **上下客热点区域**：将起终点经纬度四舍五入至 0.001°（约 111 m），按网格聚合后取 Top 20。上车最密集区域为罗湖火车站附近（22.524, 114.068, 4,038 单），下车最密集区域同样在罗湖火车站片区（22.522, 114.068, 2,769 单）。
- **最终输出**：stats_summary.json 约 5.7 KB，封装了所有聚合统计结果。
- 随机抽查验证：
 - 随机选取 5 辆车，从 vehicles.json 读取偏移量后截取 vehicle_data.json 对应片段，与 clean.csv 中该车辆逐行对比，时间序列完全一致。
 - 统计缓存中的 total_orders：482,398 与 wc -l orders.csv 去表头结果一致。
 - total_gps_points：38,694,096 与 clean.csv 行数一致。

(2) 问题汇报

- 订单统计中 df.groupby('hour').size() 返回的 dict 默认缺失无订单的小时（实际 24 小时均有订单，无缺失），但为保险起见使用 {str(k): int(v) for k, v in sorted(hourly_counts.items())} 确保按小时顺序输出。
- 上下客热点按 0.001° 网格聚合时，相邻网格的边界点可能被分配到不同格子导致热点分散。当前方案已能有效识别主要交通枢纽（罗湖口岸、深圳北站、宝安机场周边），后续若需求更平滑的热区可改用 DBSCAN 聚类后再聚合。

团队对成员本日贡献的自我评价：

成员	评分	等级	评价理由
成员A (组长)	94	优	独立完成 OD 异常过滤与 2.6 GB 车辆缓存构建的全部编码，流式 JSON 写入方案设计合理，跨 chunk 拼接逻辑正确，产出数据经完整性校验通过
成员B	90	优	统计缓存实现涵盖订单分布、载客率、速度、热点四类聚合，数据准确；缓存验证环节设计了三项交叉检查，确保上下游数据一致性

评分说明：两位成员各自独立完成了缓存体系中不同维度的核心模块，产出数据经文件行数校验与随机抽样验证，所有数字可追溯至实际运行输出，故评为优秀等级。

验证依据：

数据项	验证方式	结果
OD 表行数	wc -l data/orders.csv 去表头	482,398
无效订单过滤	距离 ≤ 0 删除数量	1,810 条
车辆总数	vehicle_list.json 数组长度	11,068
轨迹点总量	stats_summary.json total_gps_points	38,694,096
vehicle_data.json 完整性	json.load() 试读	通过
车辆轨迹随机抽查	5 辆车逐点对比 clean.csv	一致
总订单交叉校验	orders.csv 行数 vs stats_summary	482,398 一致
车辆元数据偏移验证	按偏移读取的轨迹点顺序与 clean.csv 一致	通过